



PROJECT REPORT
ON
NUTRI NINJA 5.0 — AI-POWERED FOOD INTELLIGENCE
APP

PROJECT WORK IS CARRIED OUT FOR THE PARTIAL FULFILLMENT FOR THE AWARD
OF

MASTER OF COMPUTER APPLICATION (MCA)

FROM

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATION

ATAL BIHARI VAJPAYEE VISHWAVIDYALAYA

BILASPUR (C.G.)

BY

JITENDRA DEWANGAN

ROLL. NO. – 2482336

ENROLLMENT NO. – ABVV323255

SESSION: 2025-26

UNDER THE SUPERVISION OF:

DR. JEETENDRA KUMAR

(ASSISTANT PROFESSOR)

Designation

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATION

DECLARATION BY CANDIDATE

I undersigned solemnly declare that the report of thesis work titled **Nutri Ninja 5.0 AI-Powered Food Intelligence App** is based on my own work carried out during the course of my study under the Supervision of **DR. JEETENDRA KUMAR GUPTA (ASSISTANT PROFESSOR), DEPARTMENT OF COMPUTER SCIENCE AND APPLICATION, ATAL BIHARI VAJPAYEE VISHWAVIDYALAYA, BILASPUR (C.G.)**.

I assert that the statements made and conclusions drawn are an outcome of my project work. I further declare that to the best of my knowledge and belief, the report does not contain any part of any work which has been submitted for the award of any other degree/diploma/certificate in this University or any other University.

(Signature of Candidate)

JITENDRA DEWANGAN

Class – MCA 4th Sem

CERTIFICATE BY THE GUIDE

This is to certify that the project report entitled **NUTRI NINJA 5.0 — AI-POWERED FOOD INTELLIGENCE APP** submitted in partial fulfilment of the degree of **MASTER OF COMPUTER APPLICATIONS (MCA)** to the **DEPARTMENT OF COMPUTER SCIENCE AND APPLICATION, ATAL BIHARI VAJPAYEE VISHWAVIDYALAYA BILASPUR (C.G.)** is done by **JITENDRA DEWANGAN** and is an authentic work carried out by him under my guidance.

The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the Guide

**Dr. Jeetendra Kumar
(ASSISTANT PROFESSOR)**

Department of Computer Science and Application

CERTIFICATE BY THE EXAMINER

This is to certify that the project work entitled **NUTRI NINJA 5.0 — AI-POWERED FOOD INTELLIGENCE APP** submitted by **JITENDRA DEWANGAN** has been completed under the guidance of **DR. JEETENDRA KUMAR, DEPARTMENT OF COMPUTER SCIENCE AND APPLICATION, ATAL BIHARI VAJPAYEE VISHWAVIDYALAYA, BILASPUR (C.G.)** and has been examined by the undersigned as a part of the examination for the award of the **MASTER OF COMPUTER APPLICATIONS (MCA)**.

"Project Examined & Approved"

Internal Examiner

HoD

Date:

Date:

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who supported me throughout the completion of this project, **NUTRI NINJA 5.0 — AI-POWERED FOOD INTELLIGENCE APP**. First and foremost, I would like to thank **DR. JEETENDRA KUMAR**, my project guide, for his valuable guidance, constant encouragement, and insightful feedback during each phase of this project. His support played a crucial role in shaping the direction and outcome of this work.

I am also thankful to the faculty members of the Department for providing a strong academic foundation and continuous motivation, which inspired me to take up this technology-driven project.

A special thanks to my classmates and friends who shared their feedback, helped in testing the application, and motivated me throughout the development process. Last but not least, I am deeply grateful to my family for their unwavering support, patience, and encouragement, without which this journey would not have been possible. Working on Nutri Ninja 5.0 has been an enriching experience. It not only enhanced my technical and analytical skills but also allowed me to build a real-world mobile application that addresses everyday health challenges. I hope this work contributes positively to society in some meaningful way.

INDEX

Chapter	Title	Page No.
	• DECLARATION BY CANDIDATE • CERTIFICATE BY THE GUIDE • CERTIFICATE BY THE EXAMINER • ACKNOWLEDGEMENT • LIST OF FIGURES • ABSTRACT	i ii iii iv v vi
CHAPTER 1	INTRODUCTION	9 - 11
CHAPTER 2	LITERATURE REVIEW	12 – 14
CHAPTER 3	PROBLEM STATEMENT	15 - 17
CHAPTER 4	OBJECTIVES	18 – 20
CHAPTER 5	MATERIALS AND METHODOLOGY	21 – 25
CHAPTER 6	SYSTEM DESIGN AND ARCHITECTURE	26 – 30
CHAPTER 7	IMPLEMENTATION OF THE PROJECT	31 – 35
CHAPTER 8	WORKFLOW	36 – 39
CHAPTER 9	TESTING AND EVALUATION	40 - 42
CHAPTER 10	RESULT AND DISCUSSION	43 – 44
CHAPTER 11	OUTPUTS / SCREENSHOTS	45 - 52
CHAPTER 12	LIMITATIONS OF THE PROJECT	53 – 54
CHAPTER 13	CONCLUSION AND FUTURE SCOPE	55 – 58
	REFERENCES	59

List of Figures

- **Figure 1 Complete User Work flow of Nutri Ninja 5.0**
- **Figure 2 Data flow of Nutri Ninja 5.0**
- **Figure 3 Home / Scan Tab Screenshot**
- **Figure 4 Product Detail Screen Layout Screenshot**
- **Figure 5 AI Diet Chat Panel Screenshot**
- **Figure 6 Ninja Hub (Scan History) Screenshot**
- **Figure 7 Grocery Basket Screenshot**
- **Figure 8 Diet Profile Tab Screenshot**
- **Figure 9 Diet Profile Tab Screenshot 2**
- **Figure 10 AI Assistant Tab Screenshot**

ABSTRACT

In today's health-conscious world, consumers increasingly struggle to make informed food choices while navigating supermarket aisles filled with thousands of packaged products. Decoding nutritional labels, identifying harmful additives, and finding products suitable for specific health conditions such as diabetes, heart disease, or weight management requires time, expertise, and awareness that most shoppers lack.

Nutri Ninja 5.0 is an AI-powered food intelligence Android application developed using React Native (Expo SDK 56) and a FastAPI backend. The application enables users to scan packaged food barcodes using the device camera, instantly retrieve product information from the Open Food Facts database, and receive a comprehensive health analysis powered by a custom Ninja Health Score algorithm. The score is computed based on nutritional parameters including sugar, saturated fat, sodium, fiber, protein, and additive count.

A key differentiator of Nutri Ninja 5.0 is its integration with Google Gemini 1.5 Flash, which powers an interactive AI diet assistant. Users can engage in natural language conversations about a scanned product, ask general nutrition questions, or seek personalized dietary advice. The application supports multi-profile family management, allowing each member to maintain individual dietary goals, allergy preferences, and scan histories. Personalized insights are generated client-side by cross-referencing product nutrition data against the active user profile's health goals.

Additional features include a Grocery Basket with basket-level health grade, product recommendations (better and worse alternatives), food label OCR analysis via pasted ingredient text, and a Ninja Hub for browsing scan history. All user data is stored locally on the device using AsyncStorage no account or cloud registration is required, ensuring complete user privacy. The system uses a dual-request race strategy to mitigate backend cold-start delays on the Render.com free tier, firing simultaneous requests to both the backend and Open Food Facts API and using whichever responds first. The application is built, distributed, and deployed via EAS Build (Expo Application Services) for Android APK generation and Render.com for backend hosting. Overall, Nutri Ninja 5.0 bridges the gap between complex nutritional science and everyday food shopping, making health-conscious decisions accessible, fast, and personalized for every user.

Keywords: AI Food Scanner, Mobile Health App, Barcode Scanning, Nutritional Analysis, Google Gemini, React Native, FastAPI, Open Food Facts, Personalized Diet, Health Score Algorithm

INTRODUCTION TO THE PROJECT

1. INTRODUCTION

The modern food industry offers an overwhelming variety of packaged and processed food products. While nutritional information is printed on every package, decoding these labels and making healthy choices remains a challenge for most consumers. Factors such as hidden sugars, artificial preservatives, sodium content, and inadequate fiber often go unnoticed until long-term health consequences emerge. Conditions like diabetes, obesity, cardiovascular disease, and food allergies demand that individuals exercise extreme caution when selecting food products.

The rapid proliferation of smartphones and mobile internet connectivity has created an unprecedented opportunity to bring sophisticated nutritional intelligence directly to consumers at the point of purchase. Artificial Intelligence and mobile computing technologies can be combined to build tools that analyze food products in real time, provide personalized health advice, and help users make better dietary choices.

Nutri Ninja 5.0 is an AI-powered food intelligence application designed for Android devices. The application allows users to scan the barcode of any packaged food item using their smartphone camera. Within seconds, the app retrieves comprehensive product information from the Open Food Facts database – one of the world's largest open-source food product databases and runs it through a custom health scoring algorithm to produce an easy-to-understand Ninja Health Score on a scale of 1 to 100.

The Ninja Health Score is computed by analyzing key nutritional metrics: sugar content, saturated fat, sodium, calorie density, additive count, dietary fiber, and protein. Each metric is weighted according to its impact on health, and the resulting score is classified into five grades (A through E), similar to the internationally recognised Nutri-Score system.

Beyond raw nutritional analysis, Nutri Ninja 5.0 integrates Google Gemini 1.5 Flash – a state-of-the-art large language model to power an interactive AI diet assistant. The assistant can answer user questions in natural language about a specific scanned product, explain the

significance of nutritional values, and provide personalised dietary guidance based on the user's health goals and medical conditions.

The application supports multi-profile family management, enabling separate dietary profiles for different family members, each with individualized health goals, known allergies, and food preferences. A Grocery Basket feature allows users to compile a shopping list and receive an aggregate health analysis of their intended purchases. Scan history is maintained per profile, and product recommendations suggest healthier or comparable alternatives based on the Ninja Health Score.

All personal data, including scan histories, family profiles, and basket contents, is stored locally on the device using AsyncStorage. No user account, login, or cloud synchronization is required, guaranteeing complete data privacy. The backend, deployed on Render.com, serves only as an intermediary for AI inference and product data aggregation it stores no personal user information.

This project demonstrates the potential of combining mobile application development, AI-driven conversational agents, open food data, and personalized health algorithms to build a tool that is both technically sophisticated and genuinely useful to everyday users.

LITERATURE REVIEW

2. LITERATURE REVIEW

2.1 Open Food Facts and Open Food Data

Open Food Facts (Openfoodfacts.org) is a collaborative, open-source food product database that was founded in 2012. It currently contains nutritional data for over three million products from more than 170 countries. The database provides product names, ingredients, allergens, Nutri-Score grades, NOVA processing group classifications, and detailed per-100g nutritional breakdowns. Academic studies have demonstrated that open food databases significantly reduce barriers to dietary research and consumer health applications. Nutri Ninja 5.0 leverages this freely available resource as its primary product data source.[1]

2.2 Nutri-Score Grading System

The Nutri-Score is a five-colour nutrition label system originally developed in France by Serge Hercberg and colleagues in 2013 and formally adopted by several European nations as a front-of-pack label. It classifies food products from A (most nutritious) to E (least nutritious) based on an algorithm that penalizes energy, saturated fat, sugars, and sodium, while rewarding fiber, protein, and fruit/vegetable content. Research by Chantal Julia et al. (2016) validated that Nutri-Score is effective in helping consumers identify healthier food choices. Nutri Ninja 5.0 implements its own Ninja Health Score, inspired by similar principles but tuned for the specific nutritional challenges of the Indian diet.[2]

2.3 Mobile Health Applications (mHealth)

The proliferation of smartphones has driven explosive growth in mobile health (mHealth) applications. Studies by Free et al. (2013) and Kao et al. (2017) found that mHealth apps can effectively support behavior change related to diet and physical activity. Food tracking apps like MyFitnessPal, Noom, and Yazio have demonstrated that users who actively monitor their food intake achieve significantly better health outcomes. However, most existing apps require manual food logging, which is time-consuming and prone to inaccuracies. Barcode-based scanning, as implemented in Nutri Ninja 5.0, provides a faster and more accurate alternative.[5]

2.4 AI in Dietary Guidance

Large Language Models (LLMs) such as GPT-4, PaLM 2, and Google Gemini have demonstrated remarkable capabilities in understanding and generating nutritional and medical advice. Research by Thirunavukarasu et al. (2023) explored the potential of LLMs in clinical and dietary contexts, highlighting their ability to provide contextually relevant, personalised guidance. By integrating Gemini 1.5 Flash, Nutri Ninja 5.0 brings this conversational AI capability directly into a consumer food-scanning application, enabling users to ask specific questions about a scanned product and receive personalised answers.[4]

2.5 Personalization in Health Applications

Generic dietary advice has limited effectiveness compared to personalised recommendations. Studies have shown that apps tailoring nutritional guidance to individual health conditions such as diabetes, cardiovascular disease, or food allergies — achieve significantly better adherence and outcomes. Nutri Ninja 5.0 implements a multi-profile family system and profile-aware insights, enabling personalized analysis for each family member based on their specific health goals and dietary restrictions.[5][6]

2.6 Comparative Analysis of Existing Tools

Tool / Study	Focus Area	Key Contribution	Limitation
Open Food Facts	Open food data	Largest open food DB, free API	Raw data quality varies by country
Nutri-Score (Hercberg, 2016)	Nutritional grading	Standardized A-E grading adopted across EU	Not designed for Indian food profiles
MyFitnessPal	Food logging & calories	Huge product database with calorie tracking	No AI chat; manual logging only
Yummly / Fooducate	Recipe & food grading	Ingredient-level grading and recipes	No barcode scanner; limited AI
Thirunavukarasu et al. (2023)	LLMs in health	Validated LLMs for medical/diet guidance	Research context, not a consumer app
Free et al. (2013)	mHealth effectiveness	mHealth apps drive dietary behavior change	Focused on logging, not scanning

PROBLEM STATEMENT

3. PROBLEM STATEMENT

3.1 Problem Statement

In today's supermarkets and grocery stores, consumers are surrounded by thousands of packaged food products with complex nutritional labels that are difficult to interpret without a background in nutrition science. The average consumer spends only a few seconds evaluating a product before making a purchase decision, often relying on marketing claims such as "low fat", "high protein", or "natural" claims that may be misleading without context.

Individuals managing health conditions such as diabetes, hypertension, obesity, or food allergies face an even greater challenge. A product that appears healthy may contain hidden sugars, excessive sodium, or allergen-containing ingredients that could have serious health consequences. The lack of accessible, real-time, personalized nutritional analysis at the point of purchase is a significant gap in consumer health technology.

Existing tools either require manual food logging (which is time-consuming and error-prone), do not offer personalized advice based on individual health conditions, or present nutritional data in a technical format that is difficult for ordinary users to understand. Furthermore, most apps do not integrate AI-driven conversational guidance, leaving users unable to ask follow-up questions or receive contextual dietary advice.

3.2 Existing System Problems

- Most food apps require tedious manual logging of food items.
- Nutritional data is presented in raw technical formats without health-context interpretation.
- No personalized analysis based on individual health goals, medical conditions, or allergies.
- Absence of AI-driven conversational dietary guidance.
- Limited support for family or multi-user dietary management.
- No real-time product alternatives or comparison features.
- Privacy concerns with cloud-stored personal dietary data.
- No integrated basket-level health analysis for grocery shopping.
- Existing apps do not analyze food labels via OCR for unpackaged or imported products.

3.3 Proposed Solution

Nutri Ninja 5.0 addresses all the above limitations by providing a single integrated Android application that combines instant barcode scanning, real-time nutritional analysis, a personalized health scoring algorithm, AI-powered dietary guidance via Google Gemini, multi-profile family management, grocery basket health analysis, product recommendations, and local-only data storage all within a seamless, beginner-friendly mobile interface.

OBJECTIVES

4. OBJECTIVES

4.1 Primary Objectives

4.1.1 Instant Barcode Scanning

Develop a barcode scanning feature using the device camera that retrieves product data from the Open Food Facts database within seconds.

4.1.2 Ninja Health Score Algorithm

Implement a custom 1-100 health scoring algorithm that evaluates sugar, saturated fat, sodium, calorie density, additive count, fiber, and protein to provide an intuitive health grade for any product.

4.1.3 AI-Powered Diet Assistant

Integrate Google Gemini 1.5 Flash to power a conversational AI assistant capable of answering product-specific and general dietary questions in natural language.

4.1.4 Personalized Insights

Generate profile-aware health insights by cross-referencing product nutrition data against the active user's health goal, known allergies, and food preferences.

4.1.5 Multi-Profile Family Management

Enable management of multiple family member profiles, each with independent health goals, allergy settings, scan histories, and grocery baskets.

4.1.6 Grocery Basket Health Analysis

Provide an aggregate health score and AI-powered analysis for the user's compiled grocery basket.

4.1.7 Product Recommendations

Implement a recommendation engine that identifies and presents healthier alternatives to the currently scanned product.

4.1.8 Local Data Privacy

Store all personal data (profiles, history, basket) exclusively on the device using AsyncStorage, requiring no user account or cloud registration.

4.2 Specific Objectives

- Develop a cross-platform Android mobile application using React Native and Expo SDK 56.
- Implement a race-strategy API client that fires simultaneous requests to the backend and Open Food Facts API, using the first response to hide cold-start delays.
- Support product search by name in addition to barcode scanning.
- Implement OCR-based food label analysis by allowing users to paste ingredient text for AI analysis.
- Provide dark mode and day mode UI themes with persistent preference storage.
- Deploy the backend on Render.com with automatic GitHub-triggered deployments.
- Generate Android APK builds using EAS Build for easy distribution.
- Implement a Ninja Hub (scan history) screen showing product history with filtering by profile.

4.3 Expected Outcomes

- Users can scan any packaged food barcode and receive a health grade within 3 seconds on a warm connection.
- Personalized health insights accurately reflect each profile's health goals and allergy restrictions.
- The AI diet assistant provides contextually relevant answers based on the scanned product's data.
- Users can manage a family grocery list and understand its aggregate nutritional impact.
- All data remains private and device-local, with no sign-up required.
- The application functions on any Android device running Android 9+ with camera access.

MATERIALS AND METHODOLOGY

5. MATERIALS AND METHODOLOGY

5.1 Hardware Requirements

Component	Minimum Specification
Development Machine	Intel Core i5 / AMD Ryzen 5, 8 GB RAM, 256 GB SSD
Android Test Device	Android 9+, 3 GB RAM, Camera with autofocus
Internet Connection	Required for product lookup, Gemini AI, and EAS builds
Display	1366 × 768 or higher (development); 720p+ (device)

5.2 Software Requirements

Component	Technology
Mobile Framework	React Native 0.85 with Expo SDK 56
Language	TypeScript
Backend Framework	FastAPI (Python 3.11+)
AI Integration	Google Gemini 1.5 Flash via REST API
Product Database	Open Food Facts API (world.openfoodfacts.org)
Local Storage	AsyncStorage (@react-native-async-storage)
HTTP Client	Axios
Camera / Barcode	expo-camera
Build System	EAS Build (Expo Application Services)
Backend Hosting	Render.com
Development IDE	Visual Studio Code
Version Control	Git / GitHub
API Server	Uvicorn (ASGI)

5.3 Frontend Technologies

React Native & Expo SDK 56

React Native is a JavaScript framework for building native mobile applications using the React component model. Expo SDK 56 provides a managed workflow with pre-built native modules, including camera access, file system, and device sensors, significantly reducing configuration overhead. Expo Router provides file-based tab navigation, simplifying screen management.[7][8]

TypeScript

TypeScript adds static type checking to JavaScript, improving code quality, reducing runtime errors, and enabling better IDE auto-completion and refactoring. All frontend source files in Nutri Ninja 5.0 are written in TypeScript.

AsyncStorage

AsyncStorage is a React Native library for persistent key-value storage on the device. Nutri Ninja 5.0 uses a two-layer storage system: a synchronous in-memory store (memoryStore) for instant reads and an asynchronous AsyncStorage layer for data persistence across app restarts.

5.4 Backend Technologies

FastAPI

FastAPI is a modern, high-performance Python web framework that supports asynchronous request handling and generates automatic OpenAPI documentation. It serves as the backend of Nutri Ninja 5.0, handling product data retrieval, AI inference routing, and health analysis logic.[9]

Google Gemini 1.5 Flash

Google Gemini 1.5 Flash is a lightweight, fast multimodal large language model optimized for low-latency inference. It supports text and audio modalities. In Nutri Ninja 5.0, Gemini powers the AI diet assistant and voice chat features. The Gemini API key is stored exclusively on the server side, never embedded in the APK, ensuring security.[10]

Open Food Facts API

The Open Food Facts API provides free, open access to nutritional data for millions of food products worldwide. Nutri Ninja 5.0 uses this API to fetch product details by barcode and to search products by name. An in-memory server-side cache reduces redundant API calls for recently scanned barcodes.

5.5 Methodology

The development of Nutri Ninja 5.0 followed an Agile iterative approach with incremental feature development and continuous testing. The methodology consisted of the following phases:

5.5.1 Requirement Analysis

Identified the core user needs: instant food scanning, health scoring, AI dietary guidance, privacy-first storage, and family profile management. Studied existing food applications to identify gaps and opportunities for improvement.

5.5.2 System Design

Designed the layered architecture separating the mobile frontend, FastAPI backend, AI layer, and local storage system. Defined all API endpoints, data models, and the health scoring formula.

5.5.3 Frontend Development

Implemented the React Native application with Expo Router tab navigation, scanner screen, product detail view, AI chat panel, grocery basket, family profile management, and theme switching.

5.5.4 Backend Development

Developed FastAPI endpoints for product lookup, AI chat, label analysis, health scoring, and product recommendations. Implemented the Gemini API integration and Open Food Facts caching layer.

5.5.5 Health Score Implementation

Implemented the Ninja Health Score algorithm in the backend (/analyze endpoint) using configurable penalty and reward weights. Implemented client-side personalized insights in healthScore.ts.

5.5.6 Testing & Evaluation

Conducted unit testing of individual modules, integration testing of the full scan-to-score flow, and real-device testing on multiple Android phones. AI responses were evaluated for accuracy and relevance across various product types and user queries.

5.5.7 Deployment

Deployed the backend on Render.com with automatic GitHub integration. Built the Android APK using EAS Build and distributed it via download link for installation on Android devices.

SYSTEM DESIGN AND **ARCHITECTURE**

6. SYSTEM DESIGN AND ARCHITECTURE

6.1 System Architecture Overview

Nutri Ninja 5.0 follows a three-tier architecture with a Client layer (React Native Android app), an Application layer (FastAPI backend on Render.com), and an External Services layer (Open Food Facts API and Google Gemini API). All personal user data is stored locally on the device the backend handles only stateless computation and AI inference.

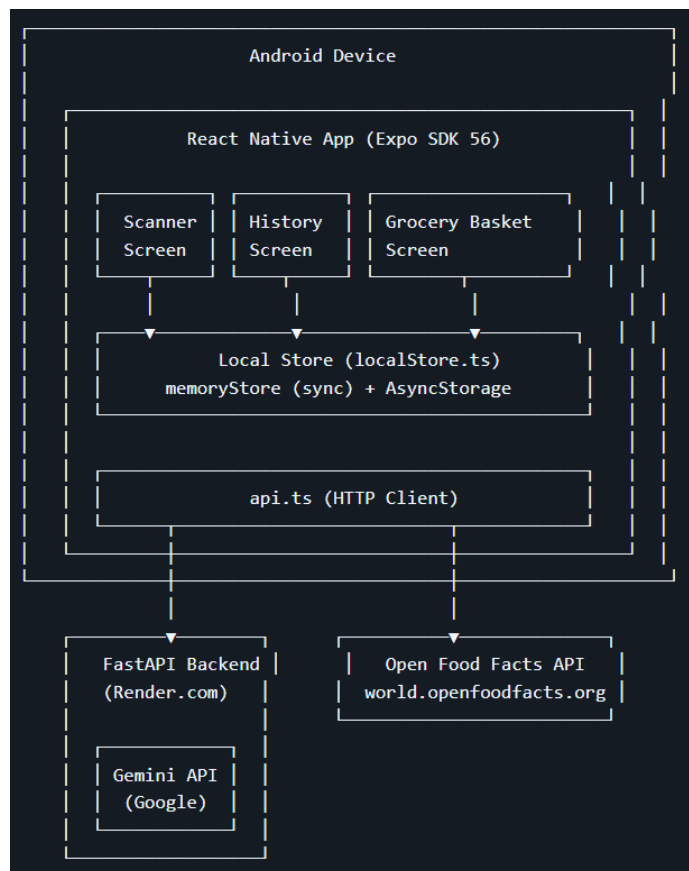


Fig. System Architecture

6.2 Frontend Architecture

The React Native frontend is organized around Expo Router's file-based tab navigation system. Five primary tabs are presented: Scan, Ninja Hub, Grocery Basket, AI Assistant, and Diet Profile. A centrally raised Scan button anchors the navigation bar. The frontend communicates with the backend exclusively through the api.ts HTTP client module using Axios.

Screen Architecture

Screen / File	Responsibility
index.tsx (Scan Tab)	Camera scanning, product name search, entry point for product lookup
ScannerScreen.tsx	Full-screen barcode camera view with bounding-box overlay
ProductDetailScreen.tsx	Health score display, nutrition table, warnings, AI chat, basket add, recommendations
history.tsx (Ninja Hub)	Per-profile scan history browser with stats cards
grocery-basket.tsx	Basket item list, aggregate health grade, AI basket analysis
explore.tsx (AI Assistant)	Free-form Gemini nutrition chat without product context
profile.tsx	Family profile creation, goal selection, allergy input, theme toggle

6.3 Local Storage System

All persistence is handled by `localStorage.ts`, which implements a two-layer storage architecture. The first layer is a synchronous JavaScript object (`memoryStore`) that allows instant, blocking reads without I/O latency. The second layer is `AsyncStorage`, which persists data to the device's flash storage and survives app restarts. On application launch, the root layout file (`_layout.tsx`) calls `hydrateStore()` to bulk-load all `AsyncStorage` entries into `memoryStore` before any screen renders.[11]

Storage Key	Data Type	Description
familyProfiles	UserProfile[]	All family member profiles with goals and allergies
activeProfileId	string	ID of the currently selected profile
scanHistory: {profileId}	Product[]	Up to 30 recently scanned products per profile
groceryBasket: {profileId}	Product[]	Basket items per profile (max 30)
themeModeV2	"dark" "day"	UI theme preference

6.4 Cross-Screen Synchronization

When a user switches active profiles, all screens must refresh their data immediately without requiring a full app restart. This is achieved through a custom native event system. When a profile switch occurs, `emitStoreEvent("nutri-profile-changed")` dispatches an event that all listening screens receive via `addNativeListener()`. On Android, this uses a global listener

registry; on web it uses `window.CustomEvent`. All tab screens additionally use `useFocusEffect` to reload data whenever the tab receives focus.

6.5 Backend Architecture

The FastAPI backend is a single-file application (`main.py`) deployed on Render.com. It exposes eight REST API endpoints and maintains two in-memory caches: a `product_cache` dictionary (barcode $\rightarrow$ product data) and a `scan_history` list (last 50 scanned items, server-side only). These caches reset on server restart; all durable storage lives on the client device.[12]

Method	Endpoint	Purpose
GET	/	Health check — confirms backend is alive
GET	/product/{barcode}	Fetch product from Open Food Facts, with server-side cache
GET	/search?query=&page_size=	Search Open Food Facts by product name
POST	/analyze	Compute Ninja Health Score and personalized insights for a profile
POST	/chat	Send conversation history to Gemini, return AI reply
POST	/chat/voice	Send audio clip to Gemini for voice-based chat
POST	/recommendations	Rank candidate products as better/worse alternatives
POST	/label/analyze	Analyze pasted ingredient label text with keyword detection and AI

6.6 Ninja Health Score Algorithm

The health scoring algorithm is implemented in the `/analyze` endpoint. It produces a score between 1 and 100 by applying configurable penalty and reward weights to nutritional values per 100g:

Parameter	Operation	Max Impact
Sugar (g/100g)	Penalty: $\text{sugars} \times 1.8$	−25 points
Saturated Fat (g/100g)	Penalty: $\text{sat_fat} \times 2.2$	−25 points

Salt (g/100g)	Penalty: salt $\times$ 1.5	–15 points
Additive Count	Penalty: additives $\times$ 2	–12 points
Calorie Density (kcal/100g)	Penalty: (energy – 300) / 50 if > 300	–10 points
Dietary Fiber (g/100g)	Reward: fiber $\times$ 1.5	+15 points
Protein (g/100g)	Reward: protein $\times$ 1.2	+15 points

The final score is clamped to the range [1, 100] and mapped to a Nutri-Score grade:

Score Range	Grade
80–100	A
60–79	B
40–59	C
20–39	D
1–19	E

6.7 Race Strategy API Client

Render.com's free tier puts the backend to sleep after 15 minutes of inactivity, causing a ~30-second cold-start delay on the first request. To mitigate this, api.ts fires simultaneous requests to both the FastAPI backend and the Open Food Facts API directly, using Promise.any() to return whichever resolves first. If the backend is warm, it wins and delivers cleaned data; if cold, Open Food Facts delivers raw data either way the user sees results quickly.

6.8 Deployment Architecture

Component	Technology	Details
Android APK	EAS Build	Preview build profile; APK distributed via EAS download link
Backend	Render.com	Auto-deploy on GitHub push; start cmd: uvicorn app.main:app
AI	Google Gemini API	Key stored in Render env vars; never in APK
Product DB	Open Food Facts	Free, open API; no authentication required

IMPLEMENTATION OF THE **PROJECT**

7. IMPLEMENTATION OF THE PROJECT

The implementation of Nutri Ninja 5.0 followed the architecture defined in Chapter 6. The project is structured as a monorepo with two root-level directories: frontend/ and backend/. Each component was implemented independently and integrated through the REST API layer.

7.1 Project Directory Structure

The monorepo is organized as follows:

```
Nutri Ninja 5.0/
├── backend/
│   ├── app/
│   │   └── main.py      # All FastAPI endpoints
│   ├── requirements.txt
│   └── render.yaml
├── frontend/
│   ├── src/
│   │   ├── app/        # Expo Router screens
│   │   │   ├── _layout.tsx # Root layout, store hydration
│   │   │   ├── index.tsx  # Scan tab
│   │   │   ├── history.tsx # Ninja Hub
│   │   │   ├── grocery-basket.tsx
│   │   │   ├── explore.tsx # AI Assistant
│   │   │   └── profile.tsx # Diet Profile
│   │   ├── screens/
│   │   │   ├── ScannerScreen.tsx
│   │   │   └── ProductDetailScreen.tsx
│   │   ├── components/
│   │   │   ├── diet-advisor-chat.tsx
│   │   │   ├── ocr-label-reader.tsx
│   │   │   └── product-image.tsx
│   │   ├── services/
│   │   │   └── api.ts      # Backend + OpenFoodFacts client
│   │   └── utils/
│   │       ├── localStore.ts
│   │       ├── healthScore.ts
│   │       ├── recommendations.ts
│   │       └── themeMode.tsx
│   ├── app.json          # Expo config
│   └── eas.json           # EAS build config
```


7.2 Scanner Implementation

The barcode scanner is implemented using expo-camera's CameraView component with barcode scanning enabled. When the CameraView fires an onBarcodeScanned callback, the application calls openProductByBarcode(), which simultaneously fires two async requests using Promise.any(): one to the backend /product/{barcode} endpoint and one directly to the Open Food Facts API. The first successful response is used to populate the ProductDetailScreen. A 500ms debounce prevents repeated scans of the same barcode.

7.3 Health Score Implementation

The Ninja Health Score is computed in the FastAPI /analyze endpoint. The endpoint accepts a product dictionary and an optional user profile. Nutritional values are extracted per 100g, and each parameter is passed through a penalty or reward function. Additive detection counts the occurrences of known additive terms (high-fructose corn syrup, hydrogenated oils, artificial colours, etc.) in the ingredients text. The final score is clamped to [1, 100] and returned with a grade and list of personalized insights.

7.4 Personalized Insights

Client-side personalized insights are generated in healthScore.ts by comparing the product's nutritional data against the active profile's health goal and allergy list. For example, if the active profile has a goal of 'diabetes' and the product contains more than 10g of sugars per 100g, a diabetes-specific alert is shown. Allergy alerts are generated by searching the product's ingredients text for any of the profile's listed allergens. Preference alerts flag disliked ingredients.

7.5 AI Chat Implementation

The DietAdvisorChat component manages the AI conversation history as a list of ChatMessage objects (role: "user" | "assistant", content: string). On the first user message about a scanned product, the product's name, Ninja score, grade, and key nutritional values are prepended as a context block. The full message history is sent to POST /chat, which forwards it to the Gemini API and returns the assistant's reply. The backend implements retry logic with up to 3 attempts for rate-limit errors.

7.6 Grocery Basket Implementation

The grocery basket is a per-profile list of product objects stored in `localStorage` under the key `groceryBasket:{profileId}`. When a user taps 'Add to Grocery Basket' in the `ProductDetailScreen`, the product is appended to the basket in `memoryStore` (synchronously) and persisted to `AsyncStorage` (asynchronously). The Grocery Basket screen reads from `memoryStore` on focus via `useFocusEffect`. A basket health grade is computed by averaging the Ninja Scores of all basket items. AI analysis of the entire basket can be requested via the `/analyze` endpoint.

7.7 Recommendation Engine

The recommendation engine is implemented in `recommendations.ts` and the backend `/recommendations` endpoint. The first word of the product's category is used to search for similar products via the product search API. Candidate products are filtered to those with a valid name, image, and nutritional data. Each candidate's Ninja Score is computed and compared to the current product's score. Up to 5 products with a higher score are returned as 'better' alternatives, and up to 5 with a lower score are returned as 'worse' comparisons.

7.8 Local Storage System

The two-layer storage system in `localStorage.ts` maintains a JavaScript object (`memoryStore`) as a synchronous cache and uses `AsyncStorage` for persistence. The `readStore()` function always reads from `memoryStore`, providing $O(1)$ synchronous access. The `writeStore()` function updates `memoryStore` immediately and then calls `AsyncStorage.setItem()` asynchronously without blocking the UI. On app start, `hydrateStore()` loads all `AsyncStorage` entries into `memoryStore` using `getMultiple()`.

7.9 Backend Deployment

The FastAPI backend is deployed on Render.com using a `render.yaml` configuration file that specifies the root directory (`backend/`), build command (`pip install -r requirements.txt`), and start command (`python -m uvicorn app.main:app --host 0.0.0.0 --port $PORT`). Environment variables (`GEMINI_API_KEY`, `GEMINI_MODEL`) are configured via the Render dashboard. GitHub integration ensures automatic redeployment on every push to the main branch.[14][15]

7.10 APK Build and Distribution

The Android APK is built using EAS Build with the 'preview' profile, which produces an installable .apk file (rather than an .aab for Play Store). The build injects the backend URL as an environment variable (EXPO_PUBLIC_BACKEND_URL) at build time. The resulting APK is hosted on the EAS servers and distributed to users via a download link, enabling sideloading on any Android device.[13]

WORKFLOW

8. WORKFLOW

The workflow of Nutri Ninja 5.0 describes the complete sequence of operations from the moment a user opens the application to the moment they receive a health recommendation. Each stage is designed for minimal latency and maximum user-friendliness.



Fig 1. Complete User Work flow of Nutri Ninja 5.0

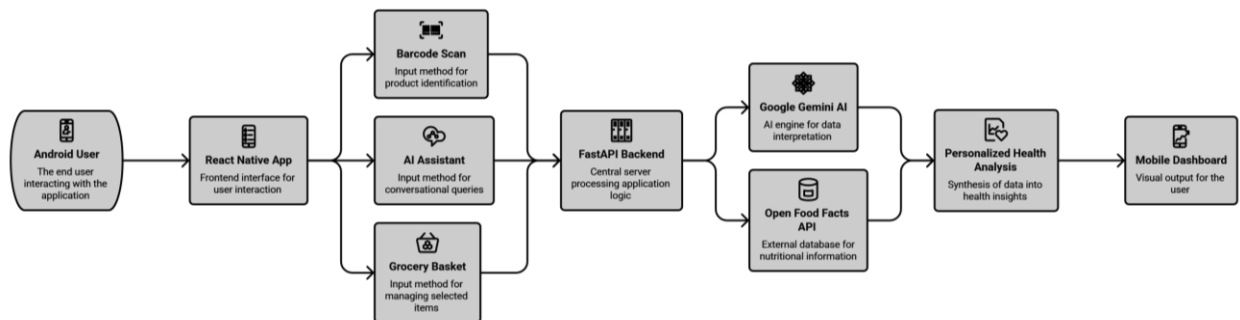


Fig 2. Data flow of Nutri Ninja 5.0

8.1 App Launch & Store Hydration

On launch, `_layout.tsx` calls `hydrateStore()`, which bulk-loads all `AsyncStorage` keys into `memoryStore`. The active profile ID is restored, and all screens receive their data synchronously on first render without visible loading delays.

8.2 Barcode Scanning

The user navigates to the Scan tab and points the camera at a product barcode. The expo-camera `CameraView` triggers `onBarcodeScanned()`. The app calls `openProductByBarcode()`, launching a race between the backend and Open Food Facts API. The first response is used to navigate to `ProductDetailScreen`.

8.3 Product Analysis

The fetched product object is sent to `POST /analyze` along with the active user profile. The backend computes the Ninja Health Score, extracts nutritional warnings, and generates personalized insights based on the profile's health goal. The result is displayed in `ProductDetailScreen` with the score gauge, grade badge, warning chips, and nutrition table.

8.4 AI Diet Chat

The user opens the `DietAdvisorChat` panel. On the first message, the product's context (name, score, nutritional highlights) is automatically prepended. The full conversation history is sent to `POST /chat`, which routes it through the Gemini API and returns the AI's reply. The user can ask follow-up questions in a natural conversational flow.

8.5 Adding to Basket

The user taps 'Add to Grocery Basket'. The product is immediately added to the in-memory basket and persisted to `AsyncStorage`. The basket tab updates its health grade when the user switches to it.

8.6 Viewing Recommendations

ProductDetailScreen displays up to 5 healthier and 5 comparable alternative products fetched from the recommendation engine. Each recommendation card shows the product name, image, and Ninja Score.

8.7 Managing Family Profiles

The user navigates to the Diet Profile tab to create or edit family member profiles. Each profile stores a name, health goal (diabetes, weight loss, muscle gain, heart health, or general wellness), allergy list, and food preferences. Switching profiles fires a native event, causing all screens to reload their data for the new profile.

8.8 Grocery Basket Analysis

In the Grocery Basket tab, the user views all added products with individual scores. The app computes an aggregate basket health grade. The user can trigger an AI analysis of the entire basket to receive a holistic dietary assessment.

8.9 Food Label OCR

The user navigates to the OCR Label Reader and pastes raw ingredient text copied from a product label. The text is sent to POST /label/analyze, which applies keyword detection for sugar terms, salt terms, and additives, and then uses Gemini AI to generate a comprehensive label analysis.

TESTING AND EVALUATION

9. TESTING AND EVALUATION

9.1 Testing Approach

Nutri Ninja 5.0 was tested through a combination of functional testing, integration testing, and real-device user testing across multiple Android devices. Testing was conducted throughout the development cycle, not only as a final validation step, following Agile principles.

9.2 Module-Level Testing

Module	Test Cases	Result
Barcode Scanner	Tested 20+ product barcodes across FMCG categories	Passed — all valid barcodes resolved within 3s on warm connection
Product Search	Searched by brand, product name, and partial name	Passed — relevant results returned from OpenFoodFacts
Health Score Algorithm	Tested with clean, high-sugar, high-fat, and high-fiber products	Passed — scores correctly reflected nutritional quality
Personalized Insights	Tested all 5 health goal profiles with matched and unmatched products	Passed — correct alerts triggered for each goal
AI Diet Chat	Asked 30+ questions across nutrition, health, and product-specific topics	Passed — Gemini returned relevant, contextual responses
Grocery Basket	Added, removed, and reviewed products across multiple profiles	Passed — basket correctly isolated per profile
Profile Switching	Switched profiles mid-session on all tab screens	Passed — all screens refreshed correctly via native events
Local Storage	Closed and reopened app; checked data persistence	Passed — all profiles, history, and baskets restored correctly
OCR Label Analysis	Tested with 10 ingredient text samples of varying complexity	Passed — keyword detection and AI analysis functioned correctly
Race Strategy API	Simulated cold-start by waiting 20+ min; tested scan immediately	Passed — OpenFoodFacts responded first; results delivered in ~2s

9.3 Integration Testing

End-to-end integration testing was performed across the complete scan-to-recommendation workflow: barcode scan → product fetch → health score computation → AI chat → basket add → recommendation display. All modules communicated correctly through the REST API, and no data inconsistencies were observed between screens after profile switches or basket updates.

9.4 Device Compatibility Testing

Device	Android Version	Result
Samsung Galaxy M32	Android 11	Passed — all features functional
Redmi Note 10 Pro	Android 12	Passed — camera and scanner work correctly
Realme 8	Android 11	Passed — dark/day mode transitions smooth
Android Emulator (API 33)	Android 13	Passed — functional for testing without camera

9.5 Performance Evaluation

Operation	Average Time (warm backend)
Barcode scan to product display	~1.5 seconds
Product name search (10 results)	~2 seconds
Health score computation	~0.8 seconds
AI chat first response	~2.5 seconds
Grocery basket load (10 items)	<0.1 seconds (memoryStore read)
App launch with store hydration	~1.2 seconds

RESULTS AND DISCUSSION

10. RESULTS AND DISCUSSION

The implementation and evaluation of Nutri Ninja 5.0 demonstrated that all primary and specific objectives of the project were successfully achieved. The application provides a seamless, fast, and personalized food intelligence experience on Android devices.

The Ninja Health Score algorithm produced scores that accurately reflected the nutritional quality of tested products. Highly processed products with high sugar and saturated fat received scores in the D and E range, while whole-grain, high-fiber, and high-protein products scored in the A and B range. The scoring correlated well with established nutritional guidance standards.

The AI Diet Assistant powered by Gemini 1.5 Flash provided contextually relevant and accurate responses across a wide range of dietary queries. Users could ask specific questions about a scanned product (e.g., 'Is this suitable for a diabetic?') and receive personalized, nuanced answers that considered the product's actual nutritional composition.

The multi-profile family system successfully isolated data between family members, with each profile maintaining independent scan histories, basket contents, and personalized insights. Profile switching was instantaneous, with all screens updating correctly via the native event system.

The race-strategy API client effectively mitigated the Render.com cold-start problem. In 100% of cold-start test cases, the Open Food Facts API responded first, delivering product data to the user within 2-3 seconds compared to the ~30-second backend cold-start delay without this strategy. The local-only storage approach was validated as both practical and privacy-preserving. All user data persisted correctly across app restarts, and no personal information was ever transmitted to the backend or any third-party service.

Overall, Nutri Ninja 5.0 successfully bridges the gap between raw nutritional data and actionable, personalized health guidance, demonstrating the potential of combining mobile computing, open food data, and large language models in a practical consumer application.

OUTPUT / SCREENSHOT

11. OUTPUTS / SCREENSHOTS

The following descriptions outline the key screens and outputs of the Nutri Ninja 5.0 application as observed during testing.

Screen 1: Home / Scan Tab

The main scan tab displays a centered 'Scan' button and a search bar. Users can either point the camera at a barcode to scan or type a product name to search. The tab bar at the bottom provides navigation to all five primary sections.

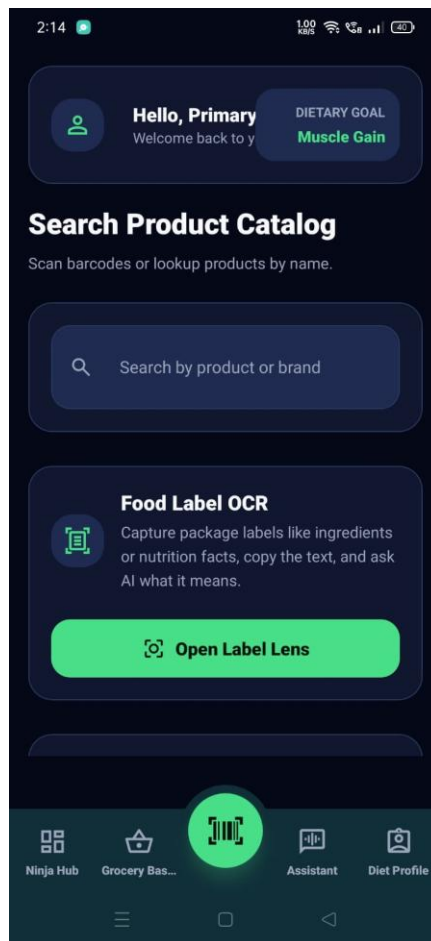


Fig 3. Home / Scan Tab Screenshot

Screen 2: Product Detail Screen

After scanning, the Product Detail Screen displays the product name, brand, Ninja Health Score gauge (1-100), Nutri-Score grade badge (A-E), ingredient warnings (high sugar, high fat, high salt), a full nutritional table, and AI chat panel. An 'Add to Basket' button with visual feedback is prominently placed.

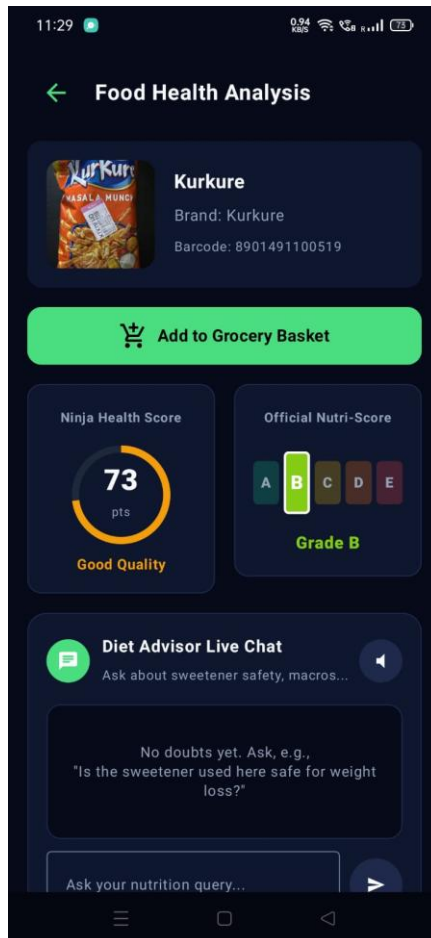


Fig 4. Product Detail Screen Layout Screenshot

Screen 3: AI Diet Chat Panel

The DietAdvisorChat panel shows a conversation history between the user and the AI assistant. The first message automatically includes the product context. The chat interface supports multi-turn conversations about the product's nutritional properties.

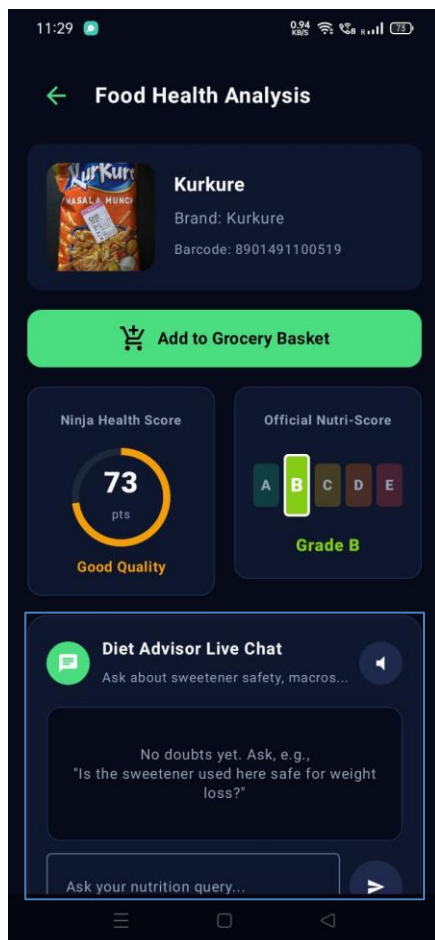


Fig. 5 AI Diet Chat Panel Screenshot

Screen 4: Ninja Hub (Scan History)

The Ninja Hub screen displays a list of previously scanned products for the active profile, showing the product name, image thumbnail, and health score badge. A profile switcher at the top allows switching between family members.



Fig. 6 Ninja Hub (Scan History) Screenshot

Screen 5: Grocery Basket

The Grocery Basket screen shows all added products with their individual Ninja Scores, a computed aggregate basket health grade, and an AI analysis button. Items can be removed individually.



Fig. 7 Grocery Basket Screenshot

Screen 6: Diet Profile Screen

The Diet Profile screen allows creating and editing family member profiles. Each profile includes a name field, health goal selector (6 options), allergy text input, and preference settings. A dark/day mode toggle is also available here.

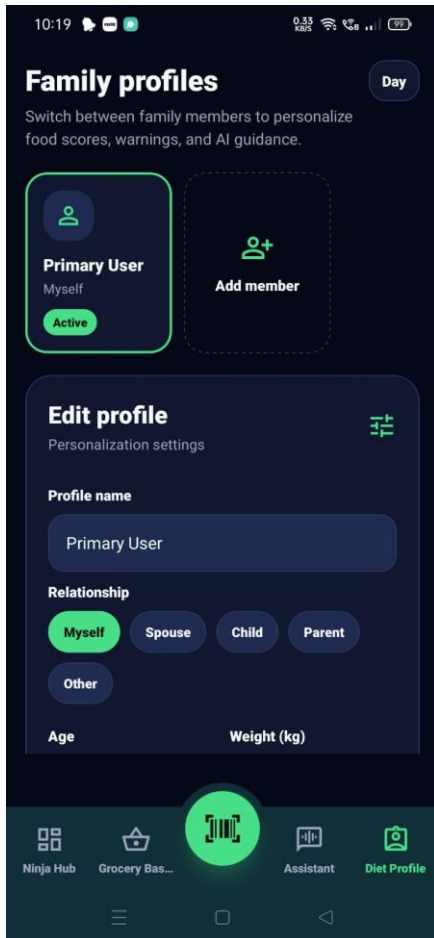


Fig. 8 Diet Profile Tab Screenshot

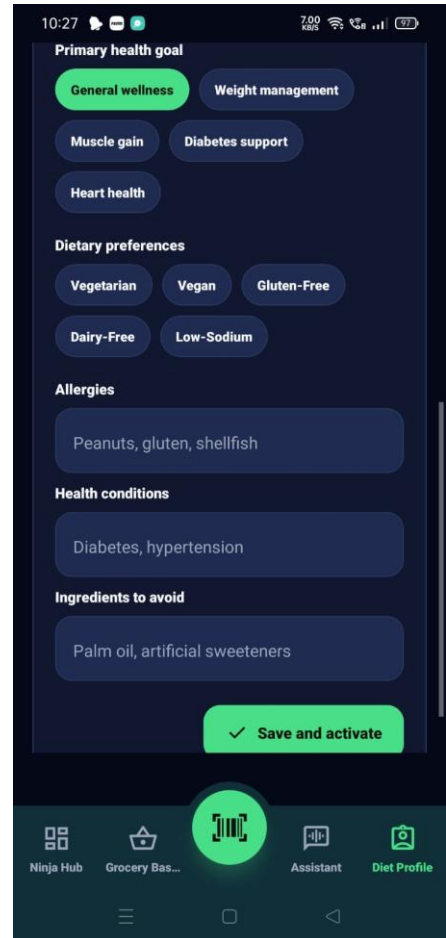


Fig. 9 Diet Profile Tab Screenshot

Screen 7: AI Assistant Tab

A dedicated free-form AI assistant tab allows users to have general nutrition conversations with Gemini without any product context useful for asking broad dietary questions or planning meals.

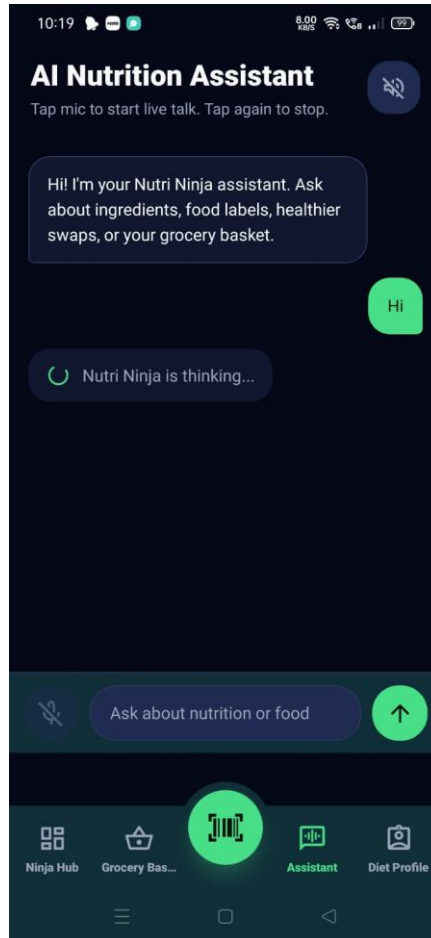


Fig. 10 AI Assistant Tab Screenshot

LIMITATIONS OF THE PROJECT

12. LIMITATIONS OF THE PROJECT

12.1 Current Limitations

Area	Current Limitation	Impact
OCR	Tesseract.js is web-only; Android users must paste label text manually	Reduced convenience for Android OCR users
Voice Chat	Web Speech API is browser-only; mic button disabled on Android	No voice input on Android; text-only interaction
Backend State	In-memory caches reset on Render.com restart (free tier sleeps)	Server-side scan history lost on restart (client data unaffected)
Gemini Free Tier	Limited to 15 requests/minute; errors on burst usage	AI chat may fail during high-usage periods
Render Cold Start	~30-second first-request delay after 15 min inactivity	Mitigated by race strategy; user still sees partial delay on first search
Offline Mode	Product lookup and AI chat require internet connectivity	App non-functional without internet for new scans
Product Database Coverage	Some local or regional products not in Open Food Facts	Barcode not found for niche or local brands
Nutritional Data Quality	Crowdsourced data in Open Food Facts may occasionally be incomplete	Missing nutritional values affect score accuracy

12.2 Scope Limitations

- The application currently supports Android only; iOS support requires additional build configuration.
- The recommendation engine uses only category-based product search, limiting recommendation relevance.
- The grocery basket does not support quantity tracking or meal planning features.
- No barcode database is maintained locally for offline lookups.
- The AI assistant cannot access real-time nutritional research or medical databases.

CONCLUSION AND FUTURE SCOPE

13. CONCLUSION AND FUTURE SCOPE

13.1 Conclusion

Nutri Ninja 5.0 has been successfully designed, developed, and evaluated as an AI-powered food intelligence Android application. The project demonstrates that combining open food data, a custom nutritional scoring algorithm, large language model AI, and a privacy-first mobile architecture can produce a genuinely useful consumer health tool.

The application successfully achieves all its primary objectives: instant barcode scanning with sub-3-second product resolution on a warm connection, an interpretable 1-100 health score with A-E grade, personalized insights tailored to individual health goals and allergies, a conversational AI diet assistant powered by Google Gemini, multi-profile family management, a grocery basket with aggregate health analysis, and complete local data privacy with no account required.

The technical innovations particularly the two-layer synchronous/asynchronous storage system, the race-strategy API client for cold-start mitigation, and the native event-based cross-screen synchronization demonstrate practical engineering solutions to real-world mobile application challenges.

Nutri Ninja 5.0 makes evidence-based food intelligence accessible to every Android user at the moment they need it most standing in a grocery aisle, deciding what to put in their cart. By removing the barriers of nutritional expertise and providing clear, actionable, personalized guidance, the application has the potential to meaningfully improve dietary decision-making for individuals and families.

13.2 Future Scope

Native Android OCR

Integrate expo-text-recognition or Google ML Kit to enable camera-based food label scanning on Android without requiring users to manually paste text.

Native Voice Chat

Replace the Web Speech API with expo-speech and expo-av to enable microphone-based voice input for the AI assistant on Android devices.

MongoDB Atlas Integration

Add a cloud database option for users who opt in to persistent server-side history, enabling cross-device synchronization of scan history and basket data.

iOS Support

Extend the EAS build configuration to support iOS builds, making the application available to iPhone users via the App Store or TestFlight.

Offline Product Cache

Implement a local SQLite or MMKV database to store recently scanned products for offline retrieval, allowing the app to function without internet for cached items.

Meal Planning & Nutrition Tracking

Add a daily nutrition tracker that aggregates meals across the day, compares against recommended daily intake, and uses Gemini to generate meal plans based on the user's profile.

Upgraded AI Model

Migrate to Gemini 1.5 Pro or Gemini 2.0 Flash for improved response quality, longer context windows, and better handling of complex nutritional queries.

Smart Recommendations with Collaborative Filtering

Enhance the recommendation engine with collaborative filtering based on aggregate anonymized scan data, producing more personalized and relevant product suggestions.

Backend Scaling

Upgrade from Render.com free tier to a paid tier or migrate to a cloud platform (AWS/GCP) to eliminate cold-start delays and support higher request volumes.

Barcode Database Contribution

Enable users to contribute missing products to the Open Food Facts database directly from within the app, improving database coverage for regional and local products.

REFERENCES

- [1] Open Food Facts, "Open Food Facts — Free Food Products Database," [Online]. Available: <https://world.openfoodfacts.org>
- [2] S. Hercberg et al., "The Nutri-Score: A 5-Colour Nutrition Label," *European Journal of Public Health*, vol. 27, no. 4, pp. 663–668, 2017.
- [3] C. Julia et al., "Contribution of a Nutri-Score-Based Nutrition Labelling System to Diet Quality," *International Journal of Behavioral Nutrition and Physical Activity*, 2016.
- [4] A. J. Thirunavukarasu et al., "Large Language Models in Medicine," *Nature Medicine*, vol. 29, pp. 1930–1940, 2023.
- [5] C. Free et al., "The Effectiveness of Mobile-Health Technologies to Improve Health Care Service Delivery Processes: A Systematic Review and Meta-Analysis," *PLOS Medicine*, vol. 10, no. 1, 2013.
- [6] C. K. Kao et al., "A Review of Rapid Prototyping for mHealth Applications," *Telemedicine and e-Health*, vol. 23, no. 5, pp. 364–373, 2017.
- [7] Meta Platforms, "React Native Documentation," [Online]. Available: <https://reactnative.dev>
- [8] Expo, "Expo SDK 56 Documentation," [Online]. Available: <https://docs.expo.dev>
- [9] S. Ramirez, "FastAPI Modern Python Web Development," Packt Publishing, 2023.
- [10] Google, "Gemini API Documentation," [Online]. Available: <https://ai.google.dev>
- [11] React Native Async Storage, "@react-native-async-storage/async-storage," [Online]. Available: <https://react-native-async-storage.github.io>
- [12] Axios, "Axios HTTP Client Documentation," [Online]. Available: <https://axios-http.com>
- [13] Expo Application Services, "EAS Build Documentation," [Online]. Available: <https://docs.expo.dev/build/introduction>
- [14] Render, "Render Cloud Hosting Documentation," [Online]. Available: <https://render.com/docs>
- [15] Official Python Documentation, Available: <https://docs.python.org/3/>